

# 多请求常驻分叉下的 vLLM Q16 同核 Attention 复用与容量分析：

## Qwen3.5-35B 4K 场景的可复现审计

匿名

2026 年 8 月 15 日

### 摘要

本文报告 Qwen3.5-35B-A3B 上的一个受控可复现实验：在单批量 (batch size=1)、Q16 精度、single-stream、同进程单 GPU 内，比较两种 same-kernel 文档所有权策略在常驻请求数  $N = \{1, 2, 4, 8, 16, 32\}$  下的解析容量与一致性。我们在 PG-19 train-only 的同一文档集合上，使用同一份约 4095-token 的预填充上下文 (worst-case partial-tail 情况)、相同几何/硬件/kernel 条件，得到两点结论：第一，两条 arm 在 token、full-vocab logits、最终 logical K/V 与 GDN state 上达到逐步一致 (token-level + full-logit exact)。第二，可复用文档策略的解析账本规模随  $N$  近似线性下降，新增一个常驻请求约节省 85 MiB，其中 80 MiB 来自避免物理文档整块复制；该优势对应的 PyTorch allocator absolute peak allocated 中位数在  $N = 32$  时下降约 2.661 GiB (约占 fresh 的 3.83%)。为避免范围漂移，本文仅讨论 **same-kernel same-batch** 条件下的解析账本与 **correctness** 门禁，不涉及吞吐/延迟、scheduler、ragged 批处理、多文档/多源泛化、Q8/Q4、下游任务指标。

#### 贡献结构。

- 在单流、同核、single-batch 条件下，给出  $N \in \{1, 2, 4, 8, 16, 32\}$  的多常驻实验与证据闭环。
- 验证同核复用与 fresh baseline 在 token、full-vocab、logical K/V、GDN state 上的 exact。
- 给出可复验的解析容量线性模型 (fresh 与 reuse) 与 allocator 中位数差异。

本文不声明并发吞吐、延迟加速、NVML 总显存提升或多文档泛化，也不声明与 HF-eager 的 logits bitwise 等价。**相关工作连接**。本实验与 transformer 注意力与其缓存优化处于同一技术谱系：Transformer 提供基础注意力机制 [2]，FlashAttention 通过块级内核优化改进内存访问 [1]，vLLM 的 paged attention 提供工程化文档块复用路径 [3]。本文不提出新 kernel，而是聚焦在该体系下的多常驻 same-kernel 复用与 exact 验证。

## 1 引言

**问题定义**。在长上下文推理里，常驻请求 (resident requests) 数目增长时，若每个请求都拷贝完整文档，解析内存会快速线性膨胀；但若共享文档并隔离请求私有区，是否能在不改变生成结果的前提下降低解析内存，是本职工作关注的核心问题。**已有差距**。前序实验已验证了 single request 时的 same-kernel exact，但没有系统扫描常驻请求数量  $N > 1$  的容量-一致性权衡。**本文贡献与边界**。本文给出 one-kernel、batch1、Q16、PG-19 train-only 下的同一文档同源跨  $N$  可比实验，提供完整证据包、审计门禁和边界。**结论边界预告**。我们只证明“解析账本改善与 correctness 保持”，并不推广到吞吐、scheduler、ragged batch、多文档、Q8/Q4 或下游任务

| 方法类                  | 共享对象                | 主要收益目标         | 本文关系                                            |
|----------------------|---------------------|----------------|-------------------------------------------------|
| prefix/past cache 共享 | 生成后缓存（通常按 token 历史） | 减少重复计算与单次前缀开销  | 我们固定同一文档并比较文档/私有账本拆分，不做跨请求调度重排                  |
| paged-kv 重用          | 文档块（KV 页池）          | 降低重建文档成本       | 本文在同一 paged 文档池下做多常驻 N 扫描与 raw 级 correctness 闭环 |
| 基于 Q16 的同核 kernel 优化 | 内核与调度策略本身           | 提升吞吐/时延或训练推理效率 | 本文不改 kernel，只验证在既定 kernel 下的解析账本规模与可复现门禁        |

表 1: 与常见方向的关系（本文不改核不改调度，仅聚焦同核多常驻解析账本）

F1/EM 改善。本文也不主张 timing/TTFT/TPOT speedup。论文组织。本文后文按以下顺序展开：方法/实验定义、正确性门禁、容量模型、可复现性治理、讨论与边界。为保证可复审性，所有量化结果默认报告 **8-rank 中位数**，并配套 per-rank raw 重放。

## 2 相关工作与定位

本文关注的切入点是：在同一份长上下文的常驻推理里，如何在 **single-stream/same-batch** 条件下，用 **same-kernel** 文档共享降低请求扩展成本，并保持输出一致性。**贡献窗口**。为保证可复现性和可辩护性，本文仅聚焦固定文档、固定调度语义下的解析账本变化；不将结果外推到跨请求调度、吞吐器优化、continuous batching、ragged 输入场景。**与本文关系**。本文与已有工程实践的差异是：我们不设计新 kernel，而是将 **同一文档多常驻请求可复用实验**与 raw 级闭环验证（token/logit/KV/GDN exact）绑定为可复用的可证伪主张。

- 与通用 KV-cache/prefix 重用类方法相比，本文不讨论跨请求调度或重排带来的吞吐收益，而是在固定文档和调度语义下，对可复用对象（document block）与私有对象（private payload）做解析级别界定。
- 与 paged KV 缓存实现相比，本文关注的不是发明新的分页机制，而是在既有统一注意力调用下，把多常驻请求场景中的共享/私有拆分转化为可追踪“解析账本容量”实验。
- 与工程级同核复用实践相比，本文在验证口径上额外加入 raw trajectory、logical K/V 与 GDN state 的 exact 重放门禁，避免“仅看最终 aggregate 统计”导致的结论偏差。

**贡献窗口**。基于上述差异，本文可表述为：给出 **fixed-one-kernel**、**same-batch**、**多常驻 N 下的解析账本与 correctness 可复验规模模型**，并明确说明结果边界。**可读性约束**。本文不讨论多文档泛化、scheduler 并发、吞吐最大化、或 Q8/Q4 与下游任务指标。**与相关文献定位**。本工作的切分是“固定调度语义下的复用规模边界”，与提出新 kernel 的方法不同；与现有文献相比，核心可复核对象在于可复现性治理与统计口径，而非算法创新本身。

### 3 方法与实验设置

**总体思路。**我们把单请求实验扩展为同一 4K 文档的常驻多请求曲线，固定几何与数据治理，同时比较两种文档所有权策略。**主命题。**在不改变输出结果的前提下，量化共享文档池方案对“常驻请求规模”是否带来可复用容量优势。**对比定义。**

- **fresh full-copy control:** 在同一 common prefill/pack 后，为每个请求分配独立 document+private 区；每个请求复制完整物理文档块。
- **shared-document reuse:** 文档块共享为只读 source arena；每请求仅持有私有页面用于 partial-tail COW 与 append。

**模型与内核。**使用 Qwen3.5-35B-A3B，40 层 Transformer，其中 full-attention 层索引为 3,7,11,15,19,23,27,31,35,39（共  $n_{\text{layer}} = 10$  层）。内核为 vLLM 0.26 unified attention，符号为 triton\_unified\_attention.unified\_attention，Q16，page size 128，batch 1。环境固定 PyTorch 2.11.0+cu129，Transformers 5.14.1，Triton 3.6.0，FlashInfer 0.6.14。

$$N \in \{1, 2, 4, 8, 16, 32\}, \quad q = 32, \quad g = 8, \quad d = 4095, \quad n_{\text{layer}} = 10$$

$$n_{\text{layer}} = 10, \quad C_{32}^{\text{layer}} = 2,621,440\text{B}$$

**单位约定。**其中  $C_{32}^{\text{layer}}$  为单层 physical page block 规模，十层 full-attn 的总规模为  $10 \times C_{32}^{\text{layer}} = 26,214,400\text{B}$ （约 25 MiB）。本文所有容量公式中的 80/90/5 MiB 均为 **10 层 full-attn 总和口径**，不是单层值。所有容量与峰值比对默认按 **8-rank 中位数聚合**报告；如使用 per-rank raw，将在文本中显式标注。

**数据与治理。**每个 rank 使用不同 PG-19 train book window；每个 book 固定冻结 32 条互不重叠 32-token query (stride=64)，无 synthetic marker，无 source6-9/68-99/test-v2 消耗，保证与生产日志链路一致。文档长度为 4095 ( $\text{mod } 128 = 127$ )，用于偏压 tail-copy 的 worst-case 应力场。**常驻规模。**每个 rank 全量运行  $N = \{1, 2, 4, 8, 16, 32\}$ ；即每个 rank 对所有 N 点都有同源 query 前缀可比（即 N 嵌套可比）。**执行顺序与资源。**8 张 H20-3e（单试验 8 卡）；每个 rank（对应单 GPU）顺序处理不同 N，但每个 rank 内 1 个 source + 2/4/8/16/32 个常驻请求对象在 setup 与生成阶段并发驻留。单流执行采用 rank 内 round-major 顺序（先各请求做 round 0，再各请求做 round 1，依次到 round 7），不构成 scheduler 并发。

### 4 正确性与一致性

**一致性约束。**每个 raw shard 先记录 token、每步 full-vocab logit、logical K/V、GDN state，并在聚合时重新计算并对照，拒绝仅信任顶层布尔量。**核心正确性结果。**

- fresh 与 reuse 在 8 个 rank、全部 N、全部请求、每个生成 token 的 full-vocab 与 token 均 exact（比例 1.0）。
- 两个 arm 的 final logical K/V 与 60-tensor GDN state exact。
- 10 个 attention 层共用同一 unified\_attention descriptor；dense fallback 与 full concat 的计数为 0。
- cross-N prefix isolation：同 rank 同一 request i 在不同 N 的输出在同一 seed/seedless 前缀下 exact，避免“共享状态污染两 arm 同时污染”的假阳性。

未能支持的外延。本实验不证明 HF-eager 与 vLLM logits/everything exact；HF 对照仅作为 non-blocking 诊断，不能替代 same-kernel 主结论。

## 5 容量与资源结果

**解析容量解析。**单层 physical page block 为 2,621,440B（十层为 26,214,400B）。在 4095 token 下, document allocation 为 32 pages, 即 80MiB；有效 payload 80MiB 减去少量 padding；每请求 private reservation 为 2 pages（约 5MiB）。**公式与解释。**

$$\text{fresh pool}(N) = 80\text{MiB} + 90N\text{MiB} \quad (1)$$

$$\text{reuse pool}(N) = 80\text{MiB} + 5N\text{MiB} \quad (2)$$

$$\text{savings}(N) = 85N\text{MiB} \quad (3)$$

其中 80MiB 来自避免的文档物理拷贝，5MiB 为每请求私有页保留，且每个请求在 active 阶段仍保留 partial-tail/append。该线性关系对应 10 层 full-attn 的账本总量聚合（ $n_{\text{full-layer}} = 10$ ），并结合文档长度 4095 的 partial-tail 最坏形态。**容量曲线（10 full-attn 层，8 rank 媒体中位数）：说明。**表里是解析账本量，不是模型总内存，也不是 NVML 总显存。每个结果均来自 raw

| N  | fresh (MiB) | reuse (MiB) | controlled savings (MiB) | avoided doc copy (MiB) |
|----|-------------|-------------|--------------------------|------------------------|
| 1  | 170         | 85          | 85                       | 80                     |
| 2  | 260         | 90          | 170                      | 160                    |
| 4  | 440         | 100         | 340                      | 320                    |
| 8  | 800         | 120         | 680                      | 640                    |
| 16 | 1520        | 160         | 1360                     | 1280                   |
| 32 | 2960        | 240         | 2720                     | 2560                   |

表 2: 解析容量中位数结果，来自 raw 重放回归后 fit。

shard 重放并带层级一致性校验。

## 6 分配器与指标解释

**统计口径。**我们使用 PyTorch allocator 的 allocated/reserved 指标，不宣称与系统级 NVML peak 等价。该指标中的“峰值/节省”默认均为 **8-rank 中位数聚合**，而 raw 对照为 per-rank 重放后再取聚合。**关键差值。**在  $N = 32$  下, absolute production peak allocated 为 fresh 74,623,183,360B、reuse 71,765,915,136B，差值 2,857,268,224B（约 2.661 GiB）。**为什么不算时延提升。**仅有单流 round-major 顺序（不是 ABBA，不是 scheduler 或 continuous batching）和 single-GPU rank 内存测量，故不报告 TTFT/TPOT/吞吐 speedup。

## 7 可复现性与治理

**身份与审计。**运行为 Job 237580 / Trial 1840837（Complete）。阶段顺序为 00\_start → 01\_static\_preflight\_ok → 02\_resident\_shards\_ok → 99\_done，无 FAILED。验证闭环。code/model/windows/manifest、PG-19 4095 windows、query bank、raw shards、ledger 均逐项 hash 复核；aggregate 不信任 shard 顶层布尔，而是重放 raw call、trajectory、allocator、storage

| N  | fresh setup+gen current delta | reuse setup+gen current delta | 差值           | fresh/reuse allocated p |
|----|-------------------------------|-------------------------------|--------------|-------------------------|
| 1  | 147.135 MiB                   | 61.883 MiB                    | 85.252 MiB   | 64.896 / 64.81          |
| 2  | 298.522 MiB                   | 126.766 MiB                   | 171.757 MiB  | 65.048 / 64.88          |
| 4  | 591.286 MiB                   | 249.520 MiB                   | 341.767 MiB  | 65.345 / 65.01          |
| 8  | 1177.831 MiB                  | 495.045 MiB                   | 682.786 MiB  | 65.936 / 65.27          |
| 16 | 2355.406 MiB                  | 990.081 MiB                   | 1365.325 MiB | 67.126 / 65.79          |
| 32 | 4705.065 MiB                  | 1980.162 MiB                  | 2724.903 MiB | 69.498 / 66.83          |

表 3: 每个值为 8-rank 中位数; reserved 主要受 allocator caching 行为影响, 本文不转化为可移植容量公式。

与 query provenance。证据链摘要。关键 SHA: summary cbb5d861...f671c0401, scientific ledger 14fd1c0c...faf363 (35/35 OK)。

## 8 讨论与局限

**可支持的结论。**在本文条件下, 可复现实验表明 reuse 策略在保持 same-kernel exact 的同时, 显著减少解析扩展, 且可线性外推到  $N=32$ 。**不应扩展的结论。**本实验不证明总显存缩放 12.3x; 不证明多文档场景、Q8/Q4、ragged-batch/continuous batching、scheduler 并发或下游任务指标。**后续方向。**下一步可在不改变 correctness 门禁的前提, 加入:

- 与 longbench/quality 的弱关联任务映射 (避免 overclaim);
- 与生产 scheduler 的 decoupled timing 评估;
- 不同文档长度与尾部分布下的 worst-case/average-case 曲线分离。

## 9 结论

**核心结论。**本文在严格 same-kernel same-batch 条件下, 给出 Q16 常驻请求复用的可复现实验:  $N$  增大时解析账本规模按模型可解释公式线性下降, 并维持 token/logit/KV/GDN exact。**实用读数。**到  $N = 32$ , 单纯解析池层面可节省 2720 MiB ( $85 \times 32$ ), 且 allocator absolute peak allocated 中位数下降约 2.661 GiB; 该值仍受缓存行为与 PyTorch 分配语义影响。**严格边界。**本工作并不推广并发吞吐, 亦不把上述账本节省直接写成全模型、NVML、或全系统级显存改善。

## 参考文献

- [1] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré, Michael L. Soffa, C. W. Chen, Noam Shazeer, Quoc V. Le, et al. Flashattention: Fast and memory-efficient exact attention with io-awareness. *arXiv preprint arXiv:2205.14135*, 2022.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

- [3] vLLM Team. vllm: High-throughput and memory-efficient inference and serving for llms. <https://github.com/vllm-project/vllm>, 2024. vLLM unified attention and paged-key-value attention implementation.